Intelligence Academy

Scikit-Learn 101

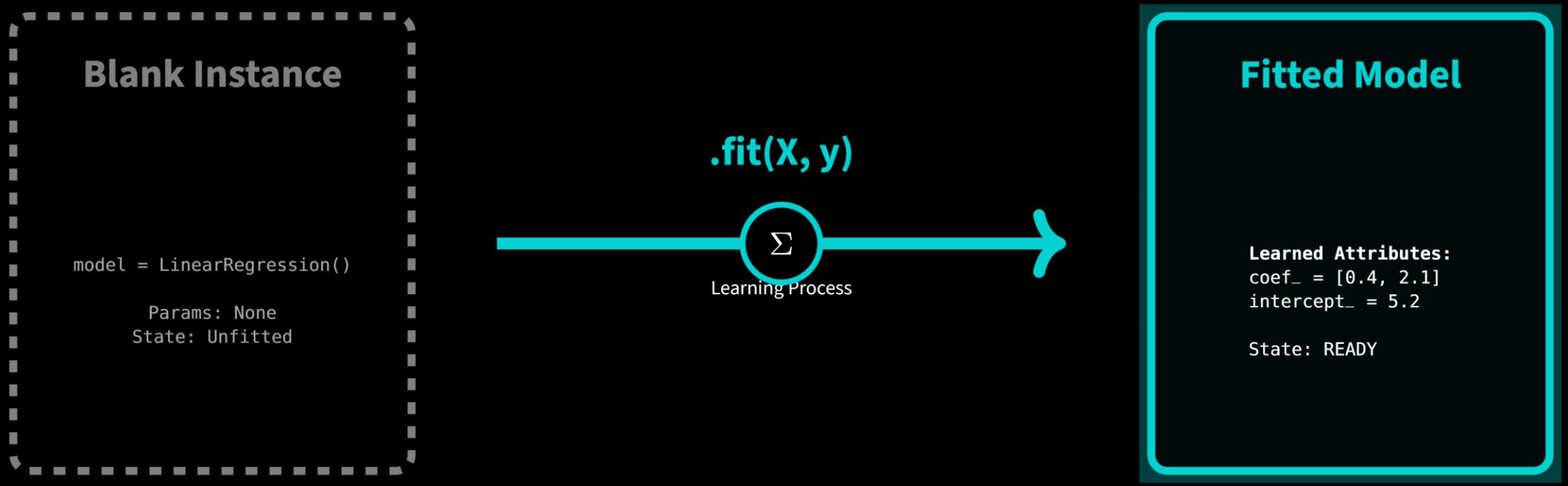
04. The lifecycle: fit → transform → predict → score

Mejbah Ahammad

1. FIT (The Foundation)

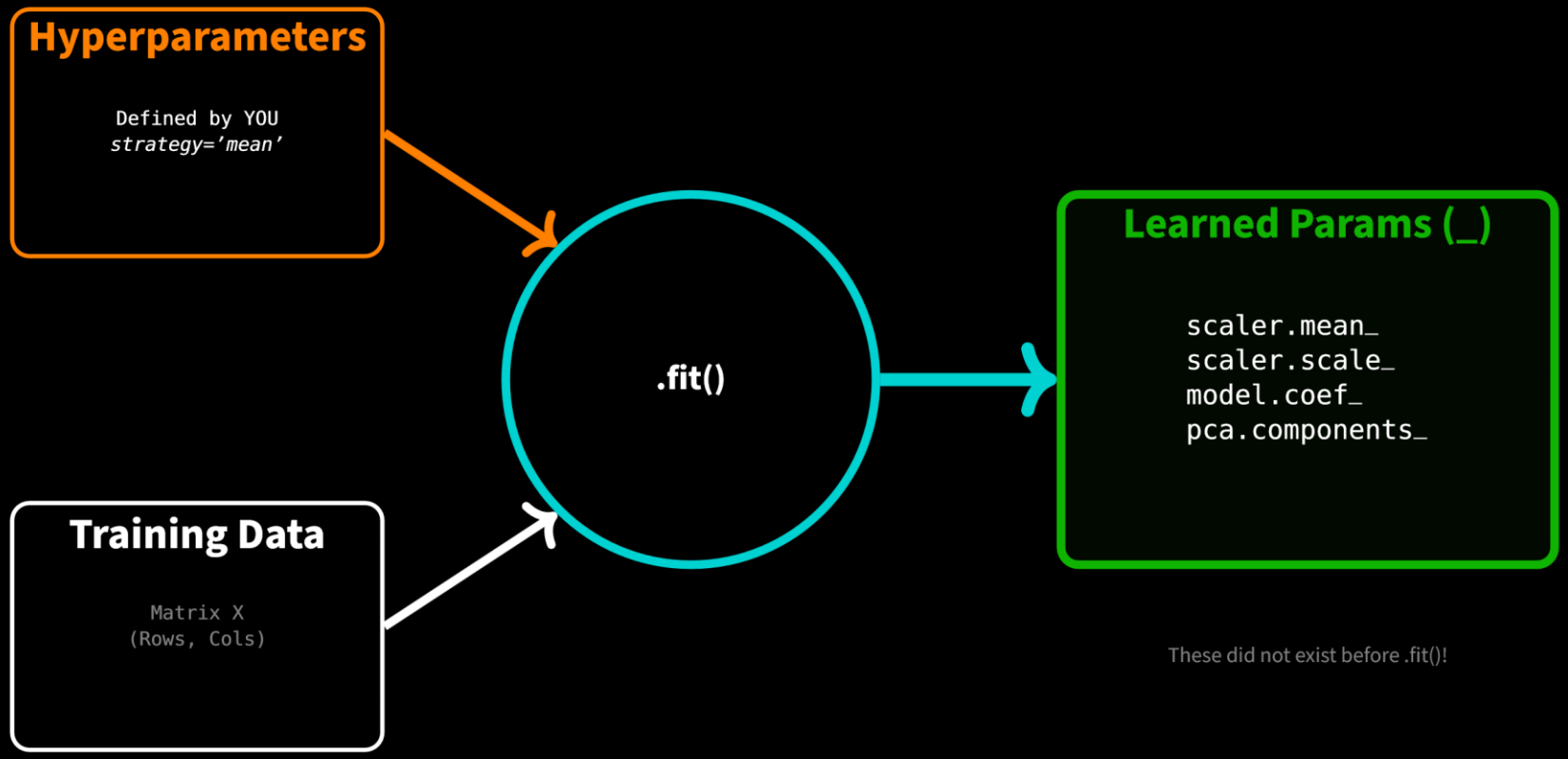
1.1 What is FIT? The State Transition

In Scikit-Learn, an estimator is born "blank." `.fit()` is the process of transitioning from a **Stateless** object to a **Stateful** one. It consumes data to populate its internal memory (Parameters).



1.2 The Mechanics: The Underscore Rule (_)

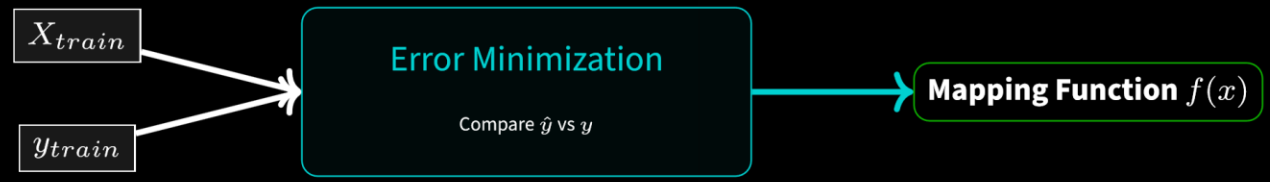
How do you distinguish "Hyperparameters" (your settings) from "Parameters" (learned knowledge)? **The Rule:** Any attribute ending in an underscore (e.g., `mean_`, `coef_`) is generated by `.fit()`.



1.3 The Two Flavors of FIT

The internal logic of `.fit()` changes entirely based on the task. **Supervised** maps Input to Output. **Unsupervised** maps Input to Structure.

A. Supervised Fit (Teacher)

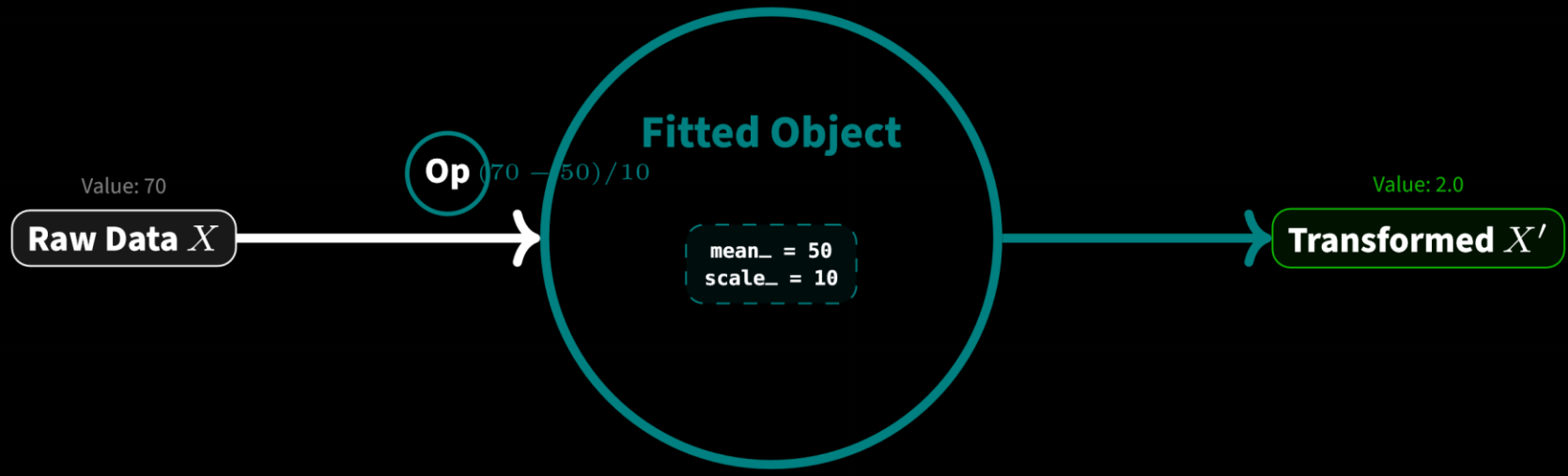


B. Unsupervised Fit (Self-Taught)

2. TRANSFORM (The Bridge)

2.1 What is TRANSFORM? The Application Phase

If `.fit()` is the Architect drawing the blueprints, `.transform()` is the Builder following them. It takes the internal parameters learned during fit (like Mean or Vocabulary) and applies them to modify new data. **Crucial:** It does not learn anything new.



Deterministic: Same Input → Same Output

2.2 The Efficiency Shortcut: fit_transform()

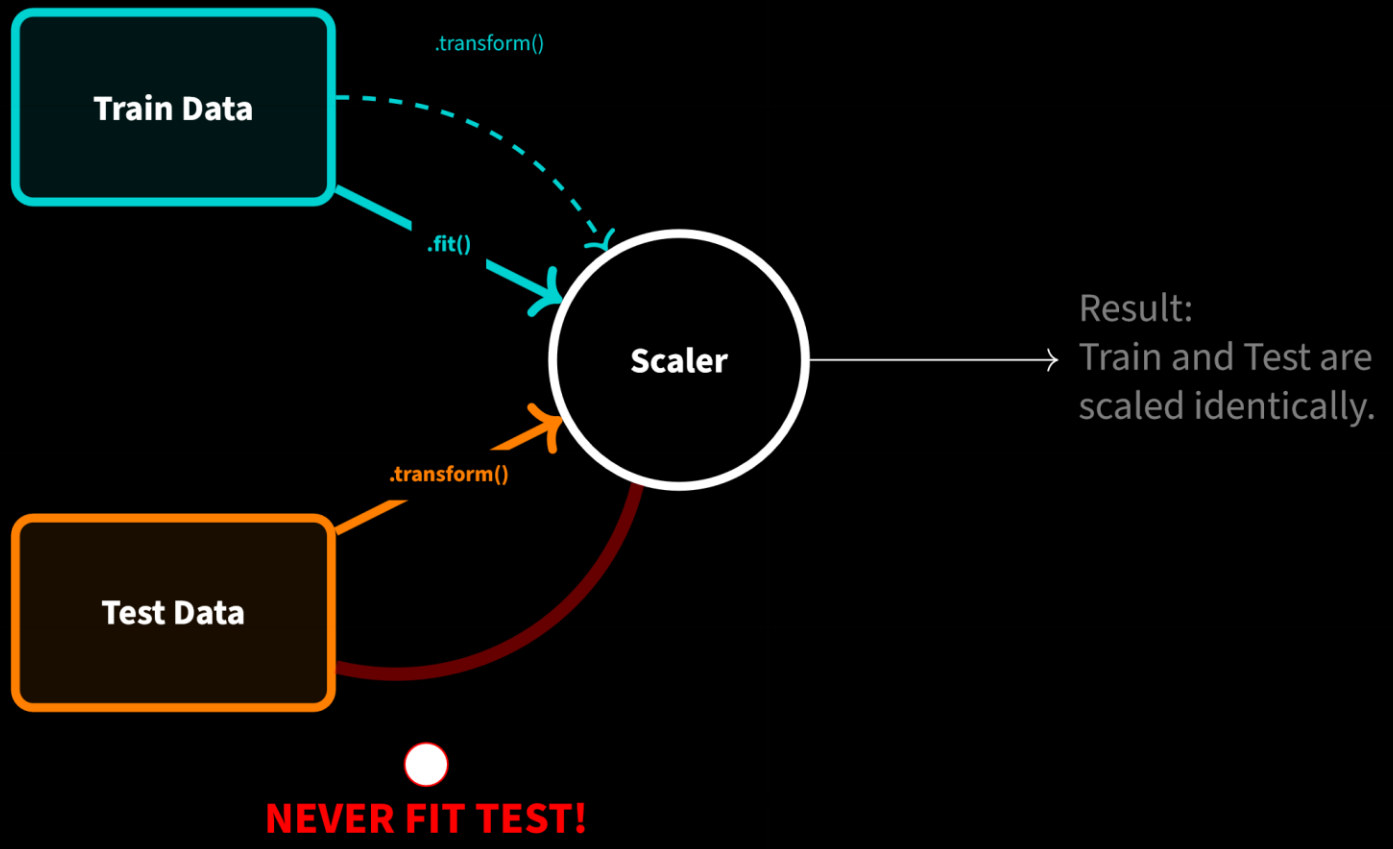
Often on training data, we need to learn the pattern AND apply it immediately. `.fit_transform()` combines these steps efficiently. **WARNING:** NEVER use this on Testing data. That is "Data Leakage."

A. The Manual Way (Two Steps)

B. The Optimized Way

2.3 The Golden Law: Train vs Test

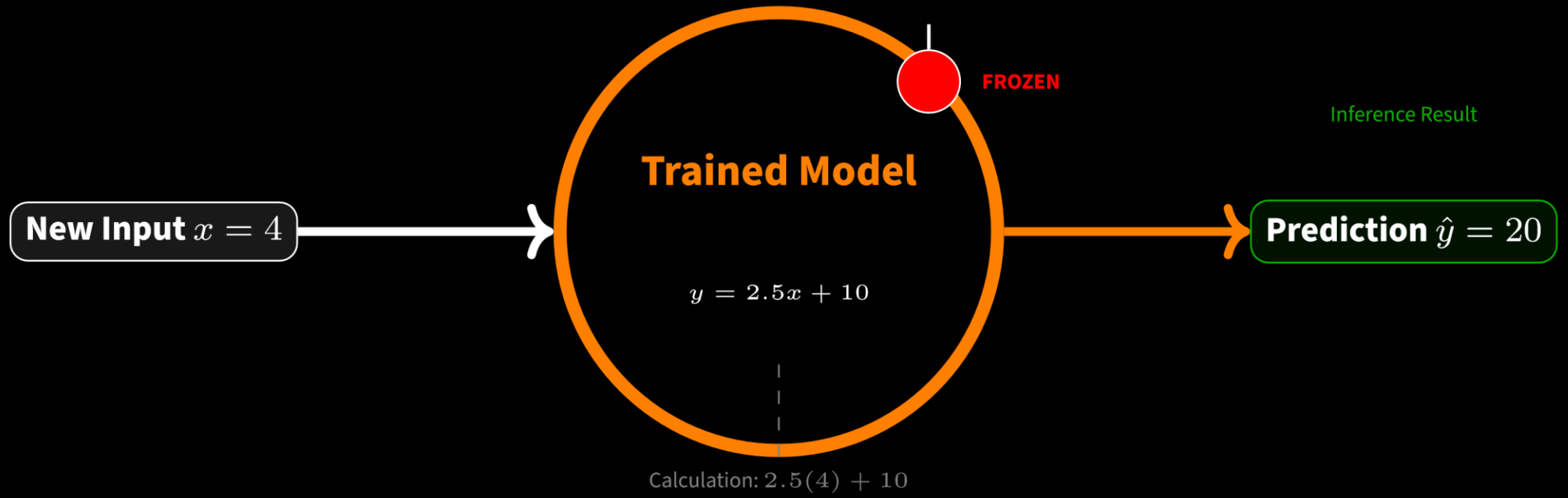
This is the most common mistake in Data Science interviews. You must **FIT** on Training data, but only **TRANSFORM** on Test data. If you fit on test data, you have cheated (Data Leakage).



3. PREDICT (The Action)

3.1 What is PREDICT? The Frozen Oracle

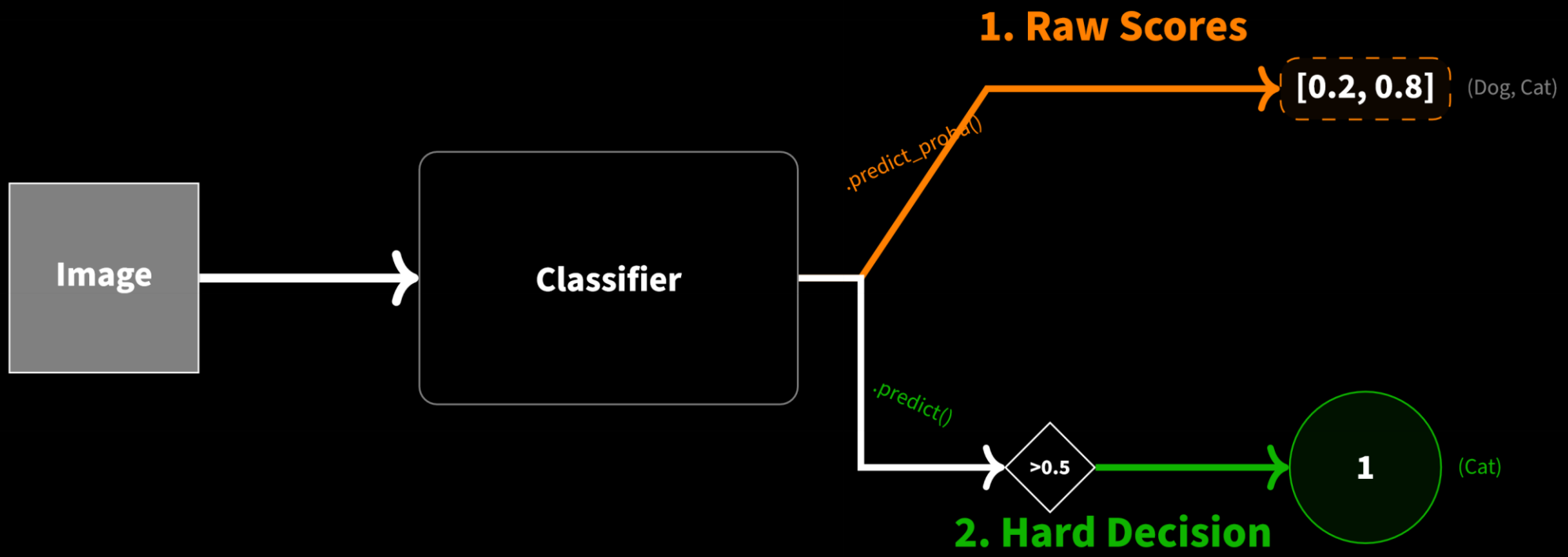
Once a model is fitted, it switches modes. `.predict()` is a **Read-Only** operation. It treats the model's parameters as constants to map new inputs (X) to outputs ($\hat{y}$). **Key:** Calling predict never changes the model.



3.2 The Decision: Probabilities vs. Labels

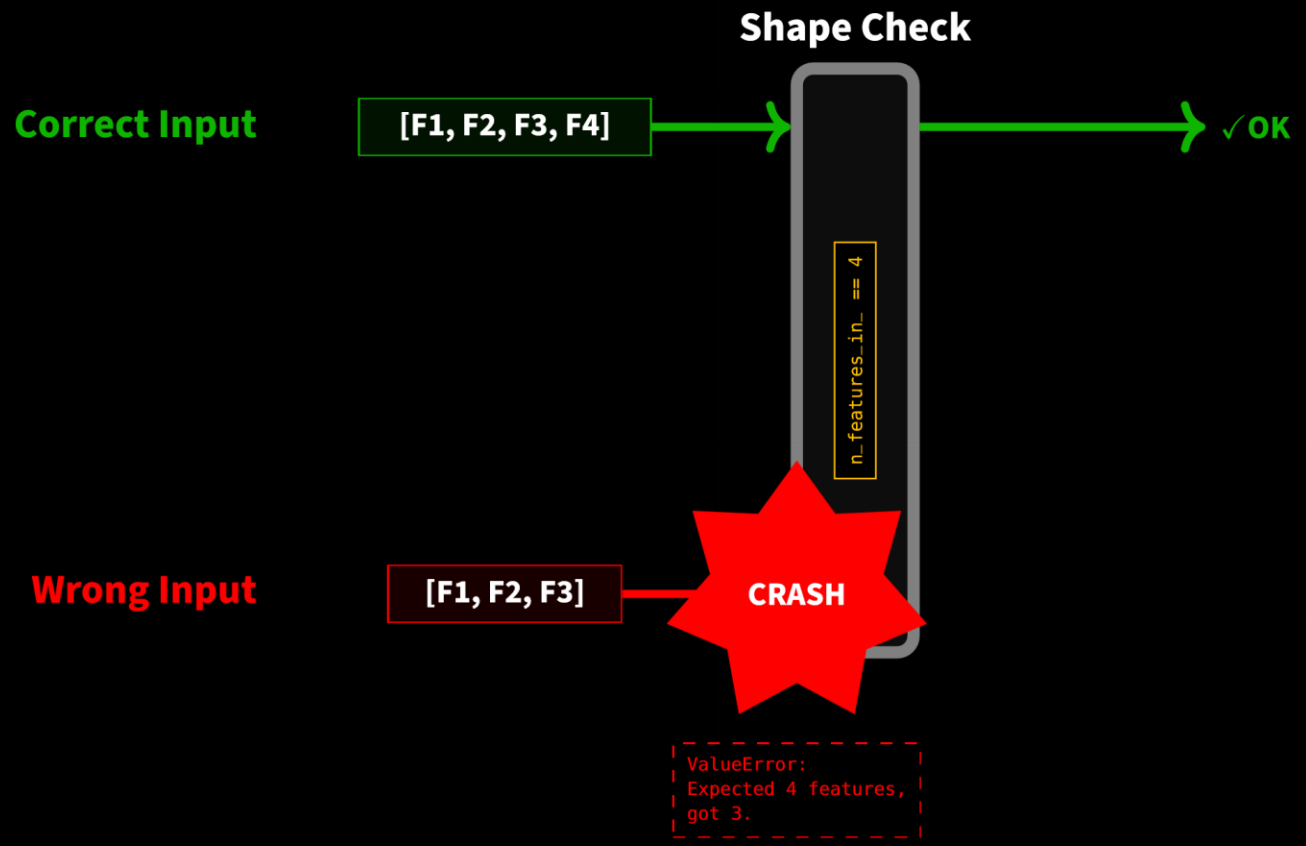
Classifiers don't just guess; they calculate **Confidence**.

- `.predict_proba()`: The raw uncertainty scores (e.g., 80% Cat, 20% Dog).
- `.predict()`: The final decision after applying a threshold (usually 50%).



3.3 The Common Error: Dimension Mismatch

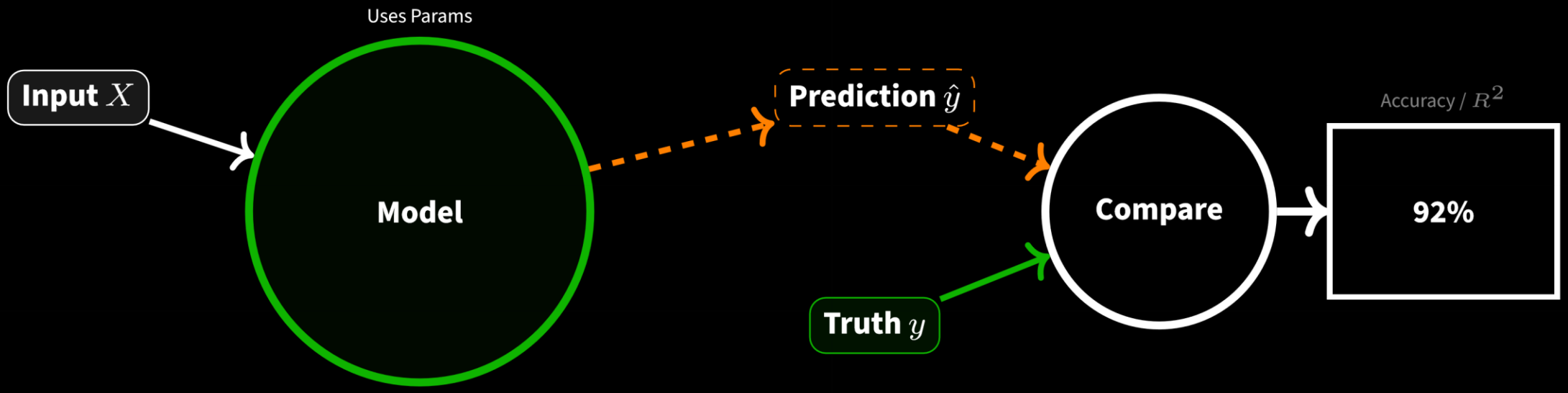
The Model remembers the **Shape** of the training data. If you trained on 4 columns (Features), you MUST predict on 4 columns. Providing 3 or 5 columns will cause an immediate crash.



4. SCORE (The Judgment)

4.1 What is SCORE? The Grading System

`.score()` is a quick way to grade your model. It condenses the model's performance into a single number (usually 0 to 1). **Requires:** Both the inputs (X) and the Truth (y) to compare against.



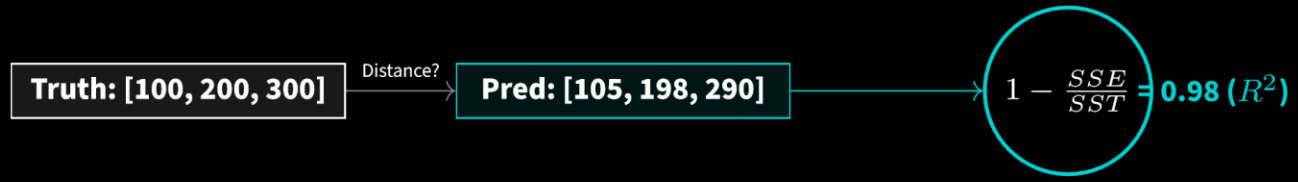
4.2 The Default Metrics: Context Matters

Calling `.score()` performs completely different math depending on the estimator type.

- **Classifiers:** Return **Accuracy** (Percent Correct).
- **Regressors:** Return R^2 (Coefficient of Determination).

A. Classification (Target is Category)

B. Regression (Target is Number)



4.3 The Overfitting Trap: Train vs. Test Score

A single score is meaningless. We must compare the **Train Score** (Memorization) against the **Test Score** (Generalization). Large gaps indicate overfitting.